

Building AI Application (Unit-IV)

1. Designing Chatbots — what, why, how (plain language)

What is a chatbot?

A chatbot is like a virtual helper you can talk to — like a receptionist who answers questions, helps place orders, or schedules meetings. It accepts text or voice, understands user intent, and replies.

Why it matters:

Chatbots automate routine tasks, give 24/7 support and can scale customer service or teaching assistants with low cost.

How it works (simple pipeline):

User input — “I want to order a pizza.”

Preprocessing — clean text, normalize (“tomor” → “tomorrow”), remove emojis if needed.

NLU (Natural Language Understanding)

Intent detection: classify “order_pizza”

Entity extraction: find “large”, “pepperoni”, “address”.

Dialogue management — decide next action (ask for address, confirm order, call API).

NLG (Natural Language Generation) — produce a response (“Got it — large pepperoni.

Deliver to 12 MG Road?”).

Action / Integration — place order, save to DB, call payment API.

Logging & Metrics — save conversation for metrics and improvement.

Simple example flow (pizza bot):

User: “I want a large pepperoni pizza.”

Bot: detects intent = order_pizza, entities = {size: large, topping: pepperoni}

Bot: “What’s the delivery address?”

After address, bot: “Confirm — Large pepperoni to 12 MG Road for ₹400. Place order? [Yes/No]”

Tiny pseudocode (conceptual)

```
message = get_user_input()
```

```
intent = classify_intent(message)
```

```
entities = extract_entities(message)
```

```
response_action = dialogue_manager(intent, entities, state)
```

```
reply = generate_response(response_action)
```

```
send(reply)
```

Building AI Application (Unit-IV)

Common pitfalls & tips

Don't overload the bot's "memory" — start with a narrow domain.

Always design fallback replies ("Sorry, can you rephrase?").

Log conversations to improve intent/entity models.

Try this: Build a rule-based bot for ordering 3 items. Use if/else to practice intent+entities.

2. Stateful Memory & Context Retention — the heart of natural chat

Plain analogy:

Stateless chat is like answering each customer as if they're a stranger. Stateful chat is like remembering that Raj ordered pizza yesterday, so you can say "Welcome back, Raj — same as last time?"

Why it matters:

Memory makes conversations feel human: follow-ups, personalization (name, preferences), and continuity across multiple turns.

Types of memory

Short-term (session): conversation history for the current session (e.g., last 5–10 messages).

Long-term (persistent): user profile, preferences, past orders saved in a database.

Episodic memory: remembered events ("You took leave on Aug 1").

Semantic memory: facts derived from interactions (user likes spicy food).

Implementation strategies

Plain session store: keep the last N messages in memory (list). Works for short chats.

Summarize & store: compress earlier conversation into a short summary and store that (keeps contexts short).

Vector embeddings + retrieval (RAG): convert saved memory into embeddings stored in a vector DB; when user asks, retrieve relevant memories and feed to model.

Key-value memory: store named facts (e.g., user_name=Raj, diet=vegetarian) in a DB.

Example: short JSON session

```
{
  "session_id": "s1",
  "history": [
    {"role": "user", "text": "Book a meeting"},
    {"role": "bot", "text": "When would you like it?"},
  ],
  "profile": {"name": "Raj", "timezone": "Asia/Kolkata"}
```

Building AI Application (Unit-IV)

}

How to use memory safely

Ask permission for long-term memory.

Provide an option to view/delete stored data.

Only store what's useful (avoid storing passwords).

Common pitfalls

Overfeeding the model with long history → slow and expensive.

Storing sensitive personal info without consent.

Forgetting to expire or delete outdated facts.

Try this: Implement a chat that remembers a user's name for the session and greets them next time in the same session.

3. Building Summarizers & Translators — simplify and cross borders

Summarizers

What is it? Turn long text into a short, readable summary.

Two approaches

Extractive: pick the most important sentences directly from text. (Like highlighting the book's key sentences.)

Abstractive: generate a new condensed version in your own words (like writing a book blurb).

How to build (simple)

For extractive: compute sentence importance (TF-IDF, TextRank) and pick top sentences.

For abstractive: fine-tune or prompt an LLM to rephrase and shorten.

Example prompt for an LLM (abstractive):

“Summarize this article in 3 short bullet points: [article text]”

Evaluation metrics

ROUGE (overlap with human summary)

Human evaluation for readability and faithfulness

Pitfall: LLMs can hallucinate — produce facts not in the original. Always verify critical summaries.

Building AI Application (Unit-IV)

Translators

What is it? Convert text from one language to another (Hindi ↔ English, etc).

How it works (brief)

Older systems used rules/statistics. Today, neural machine translation (transformers) is standard.

Large multilingual models or specialized MT models produce fluent translations.

Example usage

Use a translation model or send text to an LLM with a prompt: “Translate the following to Hindi: [text]”

Challenges

Idioms, cultural nuances, domain-specific terms.

Need for consistent terminology in technical texts.

Try this: Take one paragraph from a news article; create (a) an extractive summary and (b) an abstractive summary; compare.

4. Developing Personal AI Assistants — active helpers, not just responders

What is a personal AI assistant?

More than a chatbot — it can perform actions for you: schedule meetings, send emails, set reminders, fetch personalized summaries, or control devices.

Key components

NLU — understand requests.

Planner / Orchestrator — decides sequence of actions (e.g., check calendar → propose slots → create event).

Action connectors — APIs to calendar, email, to-do apps, smart devices.

User profile storage — preferences, frequent contacts.

Security & Auth — OAuth for calendar/email; secure tokens for APIs.

Example flow: “Schedule a meeting”

User: “Schedule a meeting with Prof. Sharma next Tuesday at 3 PM.”

Assistant: checks calendar via API (with user auth).

If free: creates event and sends invite.

If busy: suggests alternatives and asks to confirm.

Building AI Application (Unit-IV)

2. Safety & privacy

Require explicit permission for each service (Google Calendar, Gmail).

Store tokens securely and allow revocation.

Encrypt sensitive data and minimize retention.

Proactive features

Reminders (“You have a meeting in 15 minutes.”)

Suggestions (“You often meet Prof. X on Wednesdays; shall I propose a slot?”)

Try this mini feature: Build a “reminder” microservice — user asks assistant to remind at a time; the service stores reminder and triggers at due time.

5. Integration with UI Frameworks or Apps — making AI usable

Why integrate? Models are powerful but people need user-friendly interfaces: chat windows, buttons, file upload, visual summaries.

Two common frameworks

Streamlit — ideal for quick prototypes and dashboards. Minimal code to show text input, buttons, and display results.

Flask (or FastAPI) — use for building backend APIs that the frontend (web/mobile) calls.

FastAPI is faster and supports automatic docs.

Minimal Streamlit chatbot example

```
# save as app.py
```

```
import streamlit as st
```

```
st.title("Simple Chatbot")
```

```
user_input = st.text_input("You:")
```

```
if user_input:
```

```
    # call your model API or local function
```

```
    reply = my_model_respond(user_input)
```

```
    st.write("Bot:", reply)
```

Run: streamlit run app.py

Minimal Flask API endpoint (chat)

```
from flask import Flask, request, jsonify
```

```
app = Flask(__name__)
```

Building AI Application (Unit-IV)

```
@app.route('/chat', methods=['POST'])
def chat():
    data = request.json
    message = data.get('message')
    reply = my_model_respond(message)
    return jsonify({"reply": reply})

if __name__ == '__main__':
    app.run(port=5000)
```

Frontend (JS or Streamlit) calls /chat with POST JSON.

UX tips

- Show typing indicator and response latency.
- Break long answers into bullets.
- Provide clear error messages and "retry" options.
- Allow users to correct or rephrase.

Deployment basics

- Containerize with Docker.
- Host backend on cloud (Heroku, AWS, GCP) or use serverless functions.
- Use HTTPS, enable CORS properly for web clients.

Try this: Make a Streamlit front-end that calls the Flask /chat API. Test locally.

6. Mini Project Design & Planning — how to take an idea to demo

A simple project plan (4–6 weeks)

Week 1 — Idea & Requirements

- Choose domain (customer support / college assistant / translator).
- Define scope: must-have vs nice-to-have.

Week 2 — Data & Prototype

- Gather sample data or define prompts.
- Build a basic prototype: either a rule-based bot or an LLM-based responder.

Week 3 — Add Memory & Integrations

- Add session memory or small DB for user preferences.
- Integrate simple external API (calendar or email) if needed.

Building AI Application (Unit-IV)

Week 4 — UI & Testing

Build Streamlit/Flask UI.

Test with 10–20 users for usability.

Week 5 — Finalize & Deploy

Fix bugs, add logging, prepare demo and documentation.

Deploy to a cloud service or run demo locally.

Project milestones checklist

Problem statement & acceptance criteria

Data collection & privacy check

Baseline prototype (chat working)

Memory & personalization implemented

UI integrated and responsive

Tests performed & metrics collected

Final demo + README + slides

Evaluation rubric (example)

Functionality (40%): works as intended, core features complete.

Robustness (20%): handles bad input, edge cases.

UX & UI (15%): usable and clear design.

Code quality & docs (15%): readable code, comments, README.

Creativity (10%): novel idea or useful extra features.

Project ideas (simple → advanced)

Simple: Q&A bot for college notices (reads a list of notices, answers queries).

Medium: Summarizer app — upload lecture notes, get 3-bullet summary + flashcards.

Advanced: Personal assistant that schedules meetings, summarizes emails, and remembers preferences using vector memory.

Quick Practical Exercises (do these to cement learning)

- Make a rule-based chatbot for a bookstore: intents search_book, buy_book, greeting.
- Add a session memory so it remembers the user name in the conversation.
- Build a summarizer: take a news article and generate a 3-sentence summary (use an LLM or simple extractive method).
- Create a Streamlit UI that takes user text and shows the model reply.
- Mini project plan: write one page defining your project's goal, features, milestones, and evaluation criteria.

Building AI Application (Unit-IV)

Example: Rule-Based Bookstore Chatbot

This is a **working example** of a chatbot without AI (just rules).

Code

```
def bookstore_bot():

    print(" Welcome to SmartBook Store! How can I help you today?")

    while True:

        user = input("You: ").lower()

        if "hello" in user or "hi" in user:

            print("Bot: Hello! Looking for a book?")

        elif "search" in user or "find" in user:

            book = user.replace("search", "").replace("find", "").strip()

            if book:

                print(f'Bot: Let me check... Yes, we have '{book.title()}' in stock!')

            else:

                print("Bot: Please tell me the book name.")

        elif "price" in user:

            print("Bot: Most books are priced between ₹300–₹800.")

        elif "order" in user or "buy" in user:

            print("Bot: Sure! Please provide the book name and your address.")
```


Building AI Application (Unit-IV)

```
elif "bye" in user or "exit" in user:
```

```
    print("Bot: Thanks for visiting SmartBook Store. Goodbye!  ")
```

```
    break
```

```
else:
```

```
    print("Bot: Sorry, I didn't understand that. Can you rephrase?")
```

```
# Run chatbot
```

```
bookstore_bot()
```

Sample Conversation:

Welcome to SmartBook Store! How can I help you today?

You: hi

Bot: Hello! Looking for a book?

You: search harry potter

Bot: Let me check... Yes, we have 'Harry Potter' in stock!

You: price

Bot: Most books are priced between ₹300–₹800.

Building AI Application (Unit-IV)

You: order harry potter

Bot: Sure! Please provide the book name and your address.

You: bye

Bot: Thanks for visiting SmartBook Store. Goodbye!

This module covers:

- **Chatbots** (design, intents, dialogue).
- **Memory** (stateless vs stateful).
- **Summarizers & Translators** (content handling).
- **Personal Assistants** (task execution).
- **UI integration** (Streamlit, Flask).
- **Mini projects** (end-to-end design).
- **Example chatbot** (bookstore, rule-based).